PNGtoCC3Playfield

User Manual

Creating large CoCo 3 scrolling playfields

PNGtoCC3Playfield version 0.10
BasTo6809 compiler version 5.46

Glen Hewlett - August 2026

Contents

What PNGtoCC3Playfield does

Five-minute tutorial

Preparing the PNG

Choosing a PLAYFIELD

Palette workflow

Default CoCoSDC output

The -small LOADM format

Memory and block planning

Using it with BasTo6809

Current compiler integration notes

Dithering choices

Command-line reference

Troubleshooting

Project checklist

Command examples

What PNGtoCC3Playfield does

PNGtoCC3Playfield converts a large PNG picture into one of five banked CoCo 3 playfield layouts. A BasTo6809 program can select the matching `PLAYFIELD` mode, load the converted data, and use `VIEW x,y` to choose the visible 256 x 192 portion.

The converter:

- reduces the picture to one shared 16-colour CoCo 3 palette;
- packs two pixels into each byte;
- arranges 8K blocks for the selected scrolling geometry;
- creates one fast CoCoSDC streaming file by default; and
- can create ordinary numbered LOADM files with `-small`.

> **Hardware requirement:** the standard layouts begin at physical block `$80`, or 1 MB, and require a 2 MB CoCo 3. They are not CoCo 1/2 screen formats.

The five playfields trade width for height. PLAYFIELD 1 is tall and vertical; PLAYFIELD 5 is wide and horizontal.

Five-minute tutorial

1. Choose the playfield

For a simple horizontal map, begin with PLAYFIELD 2. Prepare an opaque 512 x 192 PNG named `MAP.png`.

Keep `PNGtoCC3Playfield`, `MAP.png`, and `CoCo3_Palette.asm` together in the compiler folder. The packaged program removes directory information from the input name, so keeping the files beside the executable avoids path failures.

2. Create the palette once

For the first representative project image:

```
./PNGtoCC3Playfield -p2 -makepal MAP.png
```

This creates or replaces `CoCo3_Palette.asm` and writes `MAP00.BIN`.

Afterward, omit `-makepal` so every image uses that same palette.

3. Convert later maps

```
./PNGtoCC3Playfield -p2 MAP2.png
```

The default output is already the one-file `SDC_BIGLOADM` format.

4. Load the data

In BasTo6809:

```
Playfield 2
Gmode 136, 0

SDC_BigLoadM "MAP00.BIN", 0

Screen 1, 0
View 0, 0
```

> **Loading skeleton:** a sprite-free compiler 5.46 program must also install the shared palette and commit changed `VIEW` values to the GIME registers. See Current compiler integration notes.

Preparing the PNG

Opaque RGB artwork

The program describes the input as a 32-bit RGBA PNG, but version 0.10 ignores alpha. It converts the RGB value stored under every pixel, including fully transparent pixels.

Flatten the picture onto its intended background colour before conversion. Soft transparent edges can otherwise become unexpected coloured fringes.

Dimension truncation

Before checking a playfield type, the converter truncates:

- width down to a multiple of 256; and
- height down to a multiple of 64.

It does not resize. A 780 x 205 source becomes 768 x 192 before the selected playfield rules are checked.

Prepare exact dimensions so that no artwork is silently discarded.

File names and paths

Use short, simple names such as:

```
MAP.png
LEVEL_02.png
SPACEFIELD.png
```

Put the PNG beside the executable. Passing a pathname is not reliable because version 0.10 checks the original argument and then loads only its base name. Generated `.BIN` files and `CoCo3_Palette.asm` are written in the utility's folder.

Choosing a PLAYFIELD

The following table reflects what the current converter accepts after truncation. The Recommended column narrows that to layouts that best match the stock compiler code.

Mode	Accepted source dimensions	Movement	Recommended current use
P1	256 wide; 192-7872 high; height multiple of 64	Vertical	Default big output. For `-small`, use heights divisible by 192.
P2	512 wide; 192-3840 high; height multiple of 192	Horizontal and vertical	Good general-purpose choice; default big output.
P3	1024 wide; 256-1024 high; height multiple of 256	Horizontal and vertical	Use a 1024 x 1024 source and default big output in version 0.10.
P4	512-2048 wide; width multiple of 256; height 256	Primarily horizontal, with 64 rows of vertical travel	Use default big output; current `-small` layout is not reliable.
P5	512-2560 wide; width multiple of 256; height 192	Horizontal	Use default big output. PNGtoCC3Background produces this same window family.

PLAYFIELD 3 deserves special attention. The compiler expects its three overlapping columns at physical blocks `$80`, `$A0`, and `$C0`. Version 0.10 produces that spacing correctly only when each column is 1024 pixels high and therefore occupies 32 blocks. Treat 1024 x 1024 with default big output as the currently supported PLAYFIELD 3 form.

View limits

The visible window is always 256 x 192 pixels. Keep the top-left position inside these limits:

Playfield	X range	Y range
1	0	`0` through `height - 192`
2	0 through 256	`0` through `height - 192`
3	0 through 768	`0` through `height - 192`
4	`0` through `width - 256`	0 through 64
5	`0` through `width - 256`	0

The compiler does not clamp `VIEW`. An out-of-range value can select unrelated memory. PLAYFIELD 2 through 5 use four-pixel horizontal steps, so changing only the low two bits of X has no visible effect. PLAYFIELD 1 vertical movement is pixel-granular.

Palette workflow

Build one shared palette

Use `-makepal` once on an image containing representative colours:

```
./PNGtoCC3Playfield -p2 -d1 -makepal MAINMAP.png
```

The program quantizes source colours to the CoCo 3 colour cube, selects the 16 most frequent values, and writes:

```
CoCo3_Palette.asm
```

This is a frequency selection, not a perceptual palette optimizer. A prepared reference image containing the project's desired colours may give better results than an unusually dark or monochrome level.

Reuse it everywhere

For every later image:

```
./PNGtoCC3Playfield -p2 -d1 LEVEL2.png
```

The converter reads the first 16 suitable `FCB %xxxxxxxx` lines from the existing palette. CoCo 3 sprites drawn over the playfield must also be converted with this exact file.

BasTo6809 automatically includes and installs `CoCo3_Palette.asm` when a CoCo 3 `SPRITE_LOAD` is present. `PLAYFIELD` by itself does not install the palette. A sprite-free program must write the 16 palette entries with `PALETTE` commands or equivalent assembly.

> If the palette changes, reconvert every playfield, background, and sprite that uses its colour indices.

Default CoCoSDC output

One special file

Without `-small`, PNGtoCC3Playfield creates:

```
NAME00.BIN
```

The file begins with a 512-byte table containing:

1. format version `$0001`;
2. destination physical 8K block numbers;
3. a `$FFFF` terminator; and
4. zero padding to the end of the table.

Packed 8K image blocks follow the table. This is not a DECB LOADM stream.

Load it only with:

```
SDC_BigLoadM "NAME00.BIN", 0
```

The final argument is CoCoSDC file slot 0 or 1.

Why this is the preferred format

The default file:

- loads large data through the CoCoSDC streaming interface;
- carries the physical block map with it;
- avoids managing many numbered files; and
- follows the converter's intended default path for PLAYFIELD 1, 2, 4, and 5.

For PLAYFIELD 3, use the 1024 x 1024/default-big combination described earlier.

The `-small` LOADM format

`-small` creates numbered ordinary DECB LOADM files:

```
./PNGtoCC3Playfield -p2 -small MAP.png
```

Example output:

```
MAP00.BIN
MAP01.BIN
MAP02.BIN
...
```

Each file maps one or more physical blocks into the `$4000` MMU window, loads its data, restores the normal mapping, and ends with an execute address of zero. Load all files in numerical order:

```
LoadM "MAP00.BIN:0"
LoadM "MAP01.BIN:0"
```

or:

```
SDC_LoadM "MAP00.BIN", 0
SDC_LoadM "MAP01.BIN", 0
```

Current `-small` cautions

Treat `-small` as an advanced or legacy path in version 0.10:

- PLAYFIELD 1 writes 192 rows per file even though validation permits 64-row height steps. Use a total height divisible by 192.
- PLAYFIELD 3 does not currently reproduce the compiler's required column spacing.
- PLAYFIELD 4 has MMU restoration and lower-half window-selection problems after the first horizontal window.

Use the default one-file CoCoSDC format for PLAYFIELD 3 and 4. Verify any `-small` project in MAME and on the intended machine before distributing it.

Memory and block planning

Starting block

The default is:

```
-blk128 = physical block $80 = physical address $100000
```

The stock compiler playfield routines assume this starting region. Do not change `-blk128` unless custom assembly has also been changed to match.

The utility does not check for overlap with other extended-memory assets. Plan playfields, sprites, movie buffers, audio buffers, and custom data together.

Maximum documented data use

These counts include intentional overlapping windows and seam rows:

Playfield at maximum documented size	Packed blocks	Approximate physical data
P1: 256 x 7872	126	1,032,192 bytes
P2: 512 x 3840	126	1,032,192 bytes
P3: 1024 x 1024	96	786,432 bytes
P4: 2048 x 256	56	458,752 bytes
P5: 2560 x 192	54	442,368 bytes

PLAYFIELD 1 duplicates rows 3904-4095 at its 512K video boundary. PLAYFIELD 2 duplicates rows 1856-2047. Those seam rows are intentional; they let a 192-line viewport cross the boundary cleanly.

Using it with BasTo6809

Match converter and compiler types

The number must match exactly. For example:

```
./PNGtoCC3Playfield -p2 MAP.png
```

The playfield number is a literal from 1 through 5, not a variable. The matching compiler setup, load, and initial view are:

```
Playfield 2
Gmode 136, 0
SDC_BigLoadM "MAP00.BIN", 0
Screen 1, 0
View CameraX, CameraY
```

Sprites on a playfield

For PLAYFIELD 2 through 5, convert sprites with the playfield row stride:

```
./PNGtoCCSB -g136 -scroll PLAYER.png
```

PLAYFIELD 1 is vertical-only and does not use PNGtoCCSB's `-scroll` sprite option.

All playfield and sprite conversions must share the same `CoCo3_Palette.asm`.

Current compiler integration notes

In compiler version 5.46, `VIEW` calculates mirror values named `VideoRamBlock`, `VerticalPosition`, and `HorizontalPosition`. The current sprite VBL handler performs the hardware-register writes, and its double-buffer selection is not yet a fully generic playfield display path.

Consequences:

- a sprite-free program does not automatically commit a changed `VIEW` to the GIME registers;
- sprite/playfield projects should be tested for the selected layout and buffer behavior; and
- do not treat the short tutorial as proof of flicker-free runtime integration on every playfield.

For a simple sprite-free diagnostic, the calculated mirrors can be applied after `VIEW` with inline assembly:

```
View CameraX, CameraY

' ADDASSEM:
    LDA     VideoRamBlock
    STA     $FF9B
    LDD     VerticalPosition
    STD     $FF9D
    LDA     HorizontalPosition
    STA     $FF9F
' ENDASSEM:
```

For animation, synchronize equivalent writes with vertical blank to avoid tearing. This is a current integration limitation, not a defect in the converted pixel data.

Dithering choices

Option	Method	Suggested use
`-d0`	No dither; nearest palette entry	Pixel art, flat UI, exact patterns
`-d1`	Floyd-Steinberg	General artwork and gradients; default
`-d2`	Blue-noise-style randomized mask	Textured, less directional patterning
`-d3`	Ordered pattern	Stable retro dither patterns

Dithering changes how the same 16 palette indices are distributed. It does not increase the hardware colour count.

For hand-drawn pixel art, start with `-d0`. For smooth source artwork, compare `-d1` and `-d3` in MAME and on the real video output you plan to use.

Command-line reference

```
PNGtoCC3Playfield -pN [-dN] [-makepal] [-blkN] [-small] INPUT.png
```

All options are case-insensitive and must appear before the filename.

Option	Default	Meaning
`-p1` through `-p5`	required	Select the playfield layout
`-d0`		Nearest palette colour, no dithering
`-d1`	selected	Floyd-Steinberg dithering
`-d2`		Blue-noise-style dithering
`-d3`		Ordered dithering
`-makepal`	off	Create/replace `CoCo3_Palette.asm`, then use it
`-blkN`	`-blk128`	Starting physical 8K block; N is decimal
`-small`	off	Make ordinary numbered LOADM files instead of one BIGLOADM file

There is no `-big` option. One big file is already the default.

Correct:

```
./PNGtoCC3Playfield -p2 -d0 -blk128 MAP.png
```

Incorrect:

```
./PNGtoCC3Playfield MAP.png -p2
./PNGtoCC3Playfield -p2 -blk80 MAP.png
```

The first puts the required option after the filename. The second selects decimal block 80, not hexadecimal `$80`.

Troubleshooting

The program says a required playfield was not given

Add `-p1`, `-p2`, `-p3`, `-p4`, or `-p5` before the PNG filename.

The dimensions appear different

The program truncates width to a multiple of 256 and height to a multiple of 64 before validating the selected playfield. Prepare the PNG at an exact supported size.

The PNG or palette cannot be found

Put the executable, PNG, and `CoCo3_Palette.asm` in the same folder. A path in the filename does not survive the program's basename conversion.

Colours are wrong

- Install the same 16-entry palette at runtime.
- Do not regenerate the palette between asset conversions.
- Flatten PNG transparency before conversion.
- Reconvert sprites with the same palette.

`NAME00.BIN` is not a `LOADM` file

Do not pass the default file to `LOADM`; it is a block-table stream for `SDC_BIGLOADM`, not a DECB LOADM stream:

```
SDC_BigLoadM "NAME00.BIN", 0
```

Use `-small` only when ordinary LOADM files are specifically required, and observe its current mode limitations.

`VIEW` does not visibly move the screen

Read Current compiler integration notes. In version 5.46 the command updates mirror variables; a sprite-free program needs equivalent GIME register writes.

The display points into unrelated data

- Keep `VIEW` within the range table.
- Use the same `-pN` and `PLAYFIELD N`.
- Leave the starting block at decimal 128 for the stock runtime.
- Use a 2 MB CoCo 3 and check extended-memory collisions.

Project checklist

- ☐ The target is a 2 MB CoCo 3.
- ☐ The playfield number and exact source dimensions were selected from the table.
- ☐ The image is opaque and its dimensions need no truncation.
- ☐ All command-line options precede the PNG filename.
- ☐ One representative image created the palette; every other asset reused it.
- ☐ The stock project uses decimal `-blk128`.
- ☐ The default file is loaded with `SDC_BIGLOADM`.
- ☐ Any `-small` use observes the PLAYFIELD 1, 3, and 4 cautions.
- ☐ BASIC uses matching `PLAYFIELD N` and GMODE 136.
- ☐ `VIEW` coordinates remain inside the legal range.
- ☐ PLAYFIELD 2-5 sprites were converted with PNGtoCCSB `-scroll`.
- ☐ The current view-register update path was handled and tested.

Command examples

Create a vertical PLAYFIELD 1 palette and big file:

```
./PNGtoCC3Playfield -p1 -d1 -makepal TOWER.png
```

Create a PLAYFIELD 2 file with nearest-colour mapping:

```
./PNGtoCC3Playfield -p2 -d0 MAP.png
```

Create the currently recommended PLAYFIELD 3 form:

```
./PNGtoCC3Playfield -p3 WORLD_1024X1024.png
```

Create ordinary PLAYFIELD 2 files for non-streaming loading:

```
./PNGtoCC3Playfield -p2 -small MAP.png
```

Load a default big file from CoCoSDC slot 1:

```
SDC_BigLoadM "MAP00.BIN", 1
```